TRABALHO U2 · PARTE 2 · INFRAESTRUTURA DE SOFTWARE

# Rota Vital, remodelado

DDD, arquitetura em três camadas, projeção com microsserviços e três cenários com orçamento


**Squad Hemotrack**

Ewerton Guilherme da Silva · Pablo Arthur Eustáquio de Lima · João Ricardo Alves de Brito · Lucas Aprigio dos Santos
Saulo Eduardo Almeida dos Santos · Thiago Cardozo da Conceição · Eloi de Lima Sousa

ADS · CESAR School · 2026.2

PONTO DE PARTIDA

# A Parte 1 colocou o Rota Vital no ar

### API em container

**hemocentro-api** roda como imagem Docker no Render, região Oregon.

### Banco gerenciado

**hemocentro-db** em PostgreSQL 18 no Render, disponível 24 h.

### TDD e CI/CD

Testes automatizados e esteira que gera a imagem e publica o deploy.

A Parte 2 parte desse ambiente real: o **cenário baixo** do orçamento é o que já roda hoje no Render.

A · DOMÍNIO COM DDD

# O core é entregar a bolsa certa, dentro da cadeia fria

CORE

## Atendimento de requisições

Escolher bolsas compatíveis e mais perto do vencimento, por prioridade.

## Rastreabilidade da cadeia fria

Provar que a bolsa viajou na faixa de temperatura.

SUPORTE

## Gestão de estoque

## Expedição e documentação

## Cadastro da rede

Específicos do negócio, mas sem vantagem própria: seguem procedimento padrão.

GENÉRICO

## Identidade e acesso

## Notificações

Compramos pronto: Keycloak ou Cognito; provedor de e-mail e SMS.

A · BOUNDED CONTEXTS E LINGUAGEM UBÍQUA

# Cinco contextos, cada um com sua linguagem

**Estoque**
Validade, status e reserva de bolsas

**Requisição**
Pedido, prioridade e compatibilidade (core)

**Expedição**
Separa, embala e emite a guia

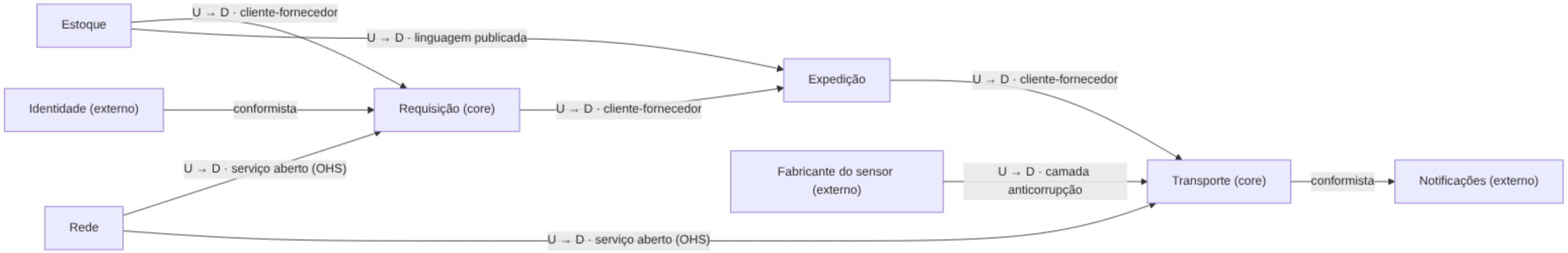
**Transporte**
Viagem, telemetria e temperatura (core)

**Rede**
Hospitais, unidades e endereços

| Termo | Contexto | Significa |
|---|---|---|
| **Bolsa** | Estoque | Unidade com doação, tipo sanguíneo, validade e status |
| **Bolsa** | Expedição | Volume a embalar: código de barras e posição na caixa |
| **Entrega** | Requisição | Hospital confirma o recebimento; encerra o pedido |
| **Entrega** | Transporte | Caixa chega ao destino; encerra a viagem |

A · MAPA DE CONTEXTOS

# O mapa acompanha o caminho da bolsa




### Cliente-fornecedor

Estoque → Requisição → Expedição → Transporte

### Anticorrupção

Adaptador traduz o sensor (°F) para nossa leitura (°C)

### Serviço aberto

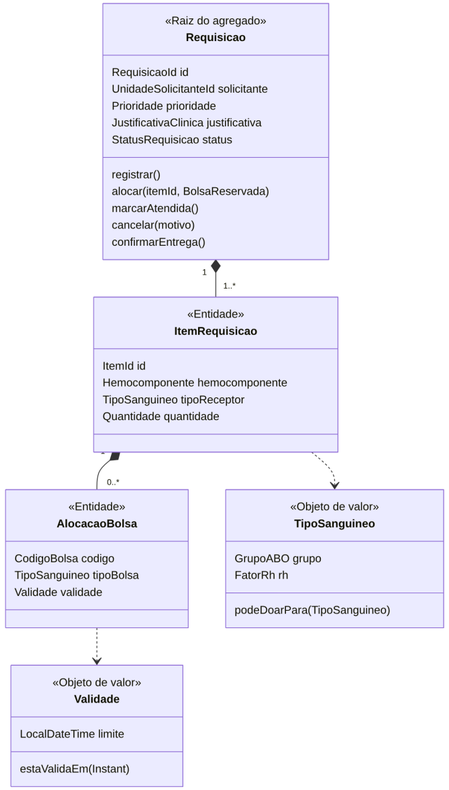
Rede expõe consulta de unidades e endereços

### Conformista

Identidade e notificações: usamos o modelo do provedor

B · MODELO TÁTICO

# O agregado Requisição barra bolsa errada




## Invariantes que a raiz protege

1. **I1 Compatibilidade:** tipo da bolsa compatível com o receptor
2. **I2 Validade:** bolsa válida no horário previsto de entrega
3. **I3 Quantidade:** nunca mais bolsas do que o pedido
4. **I4 Emergência:** exige justificativa clínica
5. **I5 Ciclo de vida:** só fica ATENDIDA com todos os itens completos

**Carla, atendente do plantão noturno · história ligada a I1 e I2**

```
Quando alocar a bolsa “B-200” do tipo “B+” ao item para receptor “A+”
Então a alocação é recusada: “Tipo sanguíneo incompatível”
```

C · TRÊS CAMADAS

# Dependências apontam só para baixo

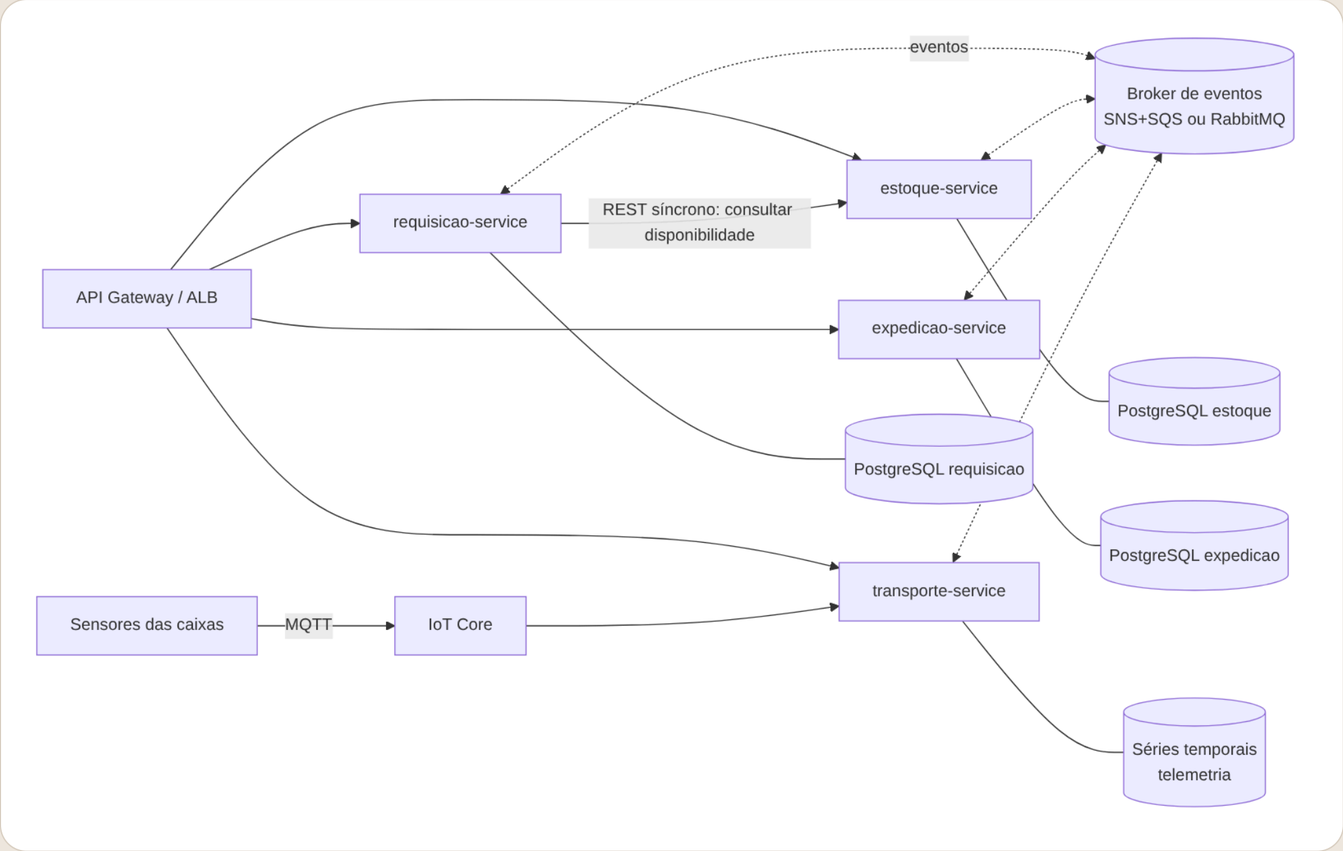



| Camada | Contém | Roda em (tier físico) |
|---|---|---|
| **Apresentação** | Controllers REST, DTOs, telas | Navegador e container da aplicação |
| **Negócio** | Casos de uso, agregados, interfaces | Container da aplicação |
| **Dados** | Repositórios JPA, integrações, fila | Container da aplicação → banco PostgreSQL |

Pacotes por contexto e por camada: br.cesar.rotavital.requisicao.negocio.dominio. Testes ArchUnit na esteira barram o domínio importando Spring, JPA ou controller.

D · PROJEÇÃO COM MICROSSERVIÇOS

# Quatro serviços, um banco para cada




## Serviços

**estoque** · **requisicao** (+ Rede) · **expedicao** · **transporte**

## Comunicação

- **REST** só para consulta na tela
- **Eventos** (SNS + SQS) com outbox
- **MQTT** para a telemetria

## Dados

Ninguém lê a tabela do outro; consistência eventual entre serviços.

D · SAGA “ATENDER REQUISIÇÃO”

# Cada passo sabe como se desfazer

| # | Serviço | Ação | Compensação |
|---|---|---|---|
| 1 | Requisição | Registra o pedido | Cancela e avisa o hospital |
| 2 | Estoque | Reserva as bolsas (30 min) | Libera a reserva |
| 3 | Requisição | Aloca e marca ATENDIDA | Desaloca, volta a EM_ATENDIMENTO |
| 4 | Expedição | Monta a remessa e a guia | Cancela a guia, devolve as bolsas |
| 5 | Transporte | Despacha (ponto sem volta) | Falha vira pedido de reposição |

O **Falha:** a Expedição acha a bolsa B-100 rompida e publica RemessaFalhou. A Requisição desfaz os passos 3 e 2, descarta a B-100 e tenta reservar de novo.

D · DECISÕES REGISTRADAS

# Monolito modular agora; Transporte sai primeiro

ADR 0001 · aceito

## Decomposição

**Contexto:** 7 pessoas, 1 esteira, pico de 3 a 120 req/s.

**Decisão:** 5 módulos em 3 camadas; Transporte extraído acima de ~50 leituras/s.

**Consequência:** menos custo e transação ACID; escala em bloco.

ADR 0002 · aceito

## Eventos com outbox

**Contexto:** salvar e publicar em dois passos pode perder o evento.

**Decisão:** o evento vai para a tabela outbox na mesma transação.

**Consequência:** nenhum evento perdido; consumidores tratam duplicatas.

E · PREMISSAS E LEI DE LITTLE

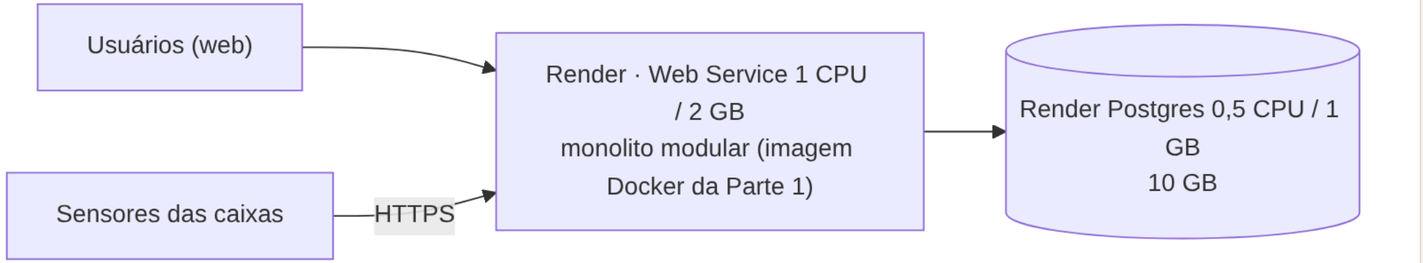
# A disponibilidade pesa mais que a carga

| | Baixo (piloto) | Moderado (PE) | Alto (Nordeste) |
|---|---|---|---|
| Hospitais · usuários | 20 · 60 | 150 · 600 | 1.200 · 5.000 |
| Pedidos por dia | 150 | 1.500 | 12.000 |
| **Pico λ** | **3 req/s** | **20 req/s** | **120 req/s** |
| Disponibilidade | 99,0% | 99,9% | 99,95% |
| **L = λ × W** | 3 × 0,20 = 0,6 | 20 × 0,20 = 4 | 120 × 0,25 = 30 |
| Instâncias (L ÷ 7) | 1 → usamos 1 | 1 → usamos 2 | 5 → 3 a 6 + 2 |
| Conexões no banco | 5 | 10 | até 70 |

No moderado, a carga cabe em 1 instância, mas **99,9% exige uma segunda em outra zona**. O banco nunca é o gargalo: no pico alto, só 6 conexões ocupadas em média.

E · CENÁRIO BAIXO · PILOTO EM RECIFE

# Uma instância e um banco no Render




Custo mensal

**US$ 47**

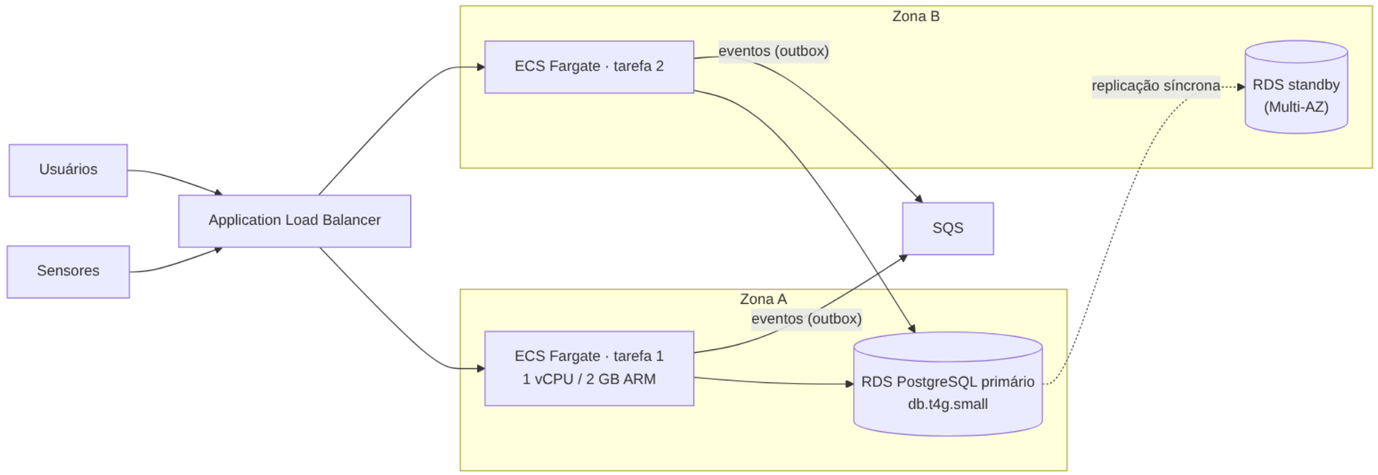
Por pedido

**US$ 0,0104**

PaaS simples: 99,0% tolera um reinício. Web Service 1 CPU / 2 GB (US$ 25) + Postgres 1 GB (US$ 19) + 10 GB de disco (US$ 3).

E · CENÁRIO MODERADO · REDE DE PERNAMBUCO

# Duas zonas na AWS para chegar a 99,9%




Custo mensal

**US$ 152,07**

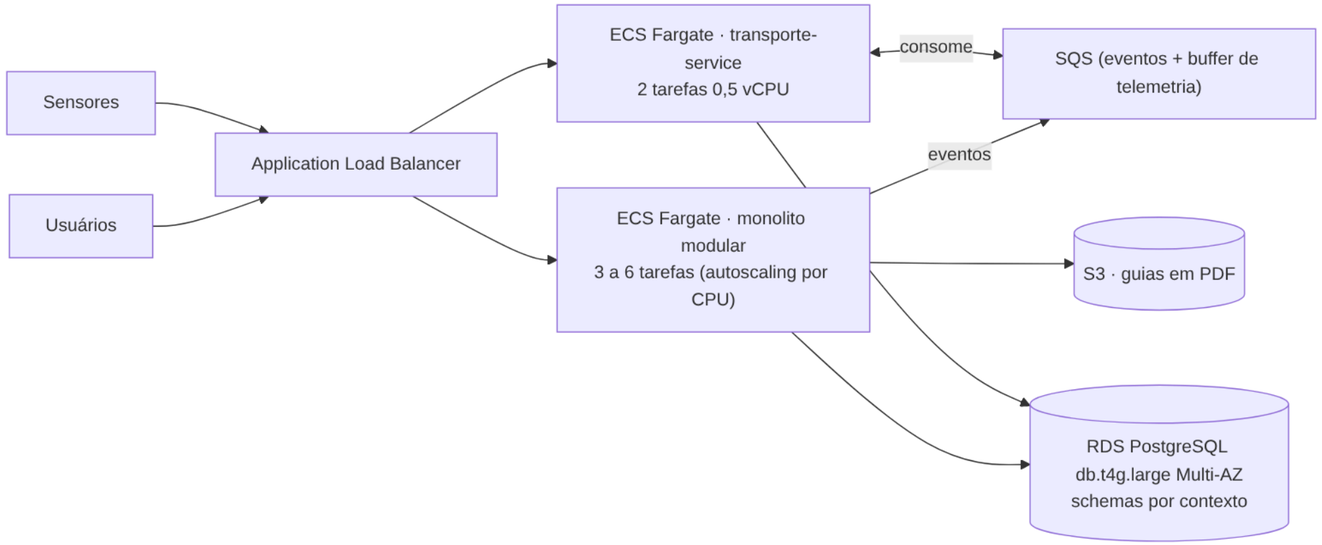
Por pedido

**US$ 0,0034**

**O que muda:** Render → AWS; 1 → 2 instâncias atrás de um load balancer; banco com standby síncrono (Multi-AZ); eventos passam a usar SQS.

E · CENÁRIO ALTO · REDE DO NORDESTE

# Autoscaling e o Transporte extraído




Custo mensal

**US$ 472,88**

Por pedido

**US$ 0,0013**

**O que muda:** 3 a 6 tarefas com autoscaling; Transporte vira serviço (ADR 0001); banco db.t4g.large; SQS absorve a telemetria; guias no S3.

E · ORÇAMENTO

# O custo sobe 10×; o custo por pedido cai 8×

Baixo · Render

**US$ 47**

4.500 pedidos · **US$ 0,0104** cada

Moderado · AWS

**US$ 152**

45.000 pedidos · **US$ 0,0034** cada

Alto · AWS

**US$ 473**

360.000 pedidos · **US$ 0,0013** cada

Custo mensal on-demand em dólar · Render (Oregon) e AWS us-east-1 · conferido pelo squad em 12/11/2026 em render.com/pricing e na AWS Pricing Calculator · detalhamento por item no documento e em orcamento.xlsx

# Perguntas?

Documento, diagramas, ADRs, planilha e cenário Gherkin no repositório:

github.com/egs3-coder/Trabalho-U2-Parte-2-Planejamento-da-remodelagem

Squad Hemotrack · próxima etapa: a Parte 3 transforma este plano em código